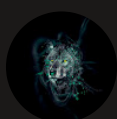
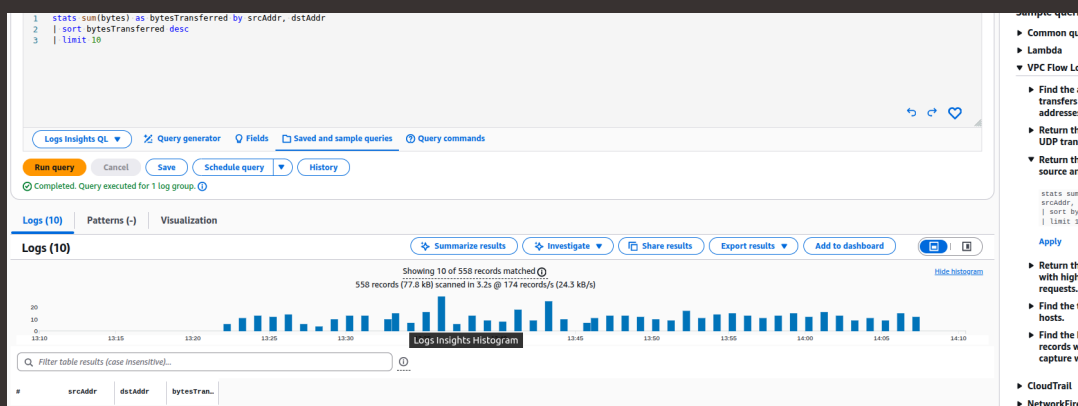


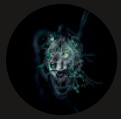


VPC Monitoring with Flow Logs



Kiprono Yegon





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service from Amazon Web Services that allows you to create a private, isolated network within the cloud where you can securely run your applications and resources. It is useful because it gives you full control over your networking environment, including IP address ranges, subnets, routing, and security settings, enabling you to protect your systems, manage traffic flow, and design scalable architectures.

How I used Amazon VPC in this project

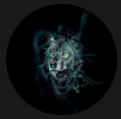
In today's project, I used Amazon VPC to monitor FLOW logs in a VPC network and visualize them with Log Insights

One thing I didn't expect in this project was...

One thing I didn't expect in this project was the use of various AWS resources that work together

This project took me...

This project took me about two hours



In the first part of my project...

Step 1 - Set up VPCs

In this step, I will create two VPCs and connect them by peering and later monitor the traffic in and out with Flow Logs and CloudWatch

Step 2 - Launch EC2 instances

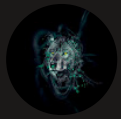
In this step, I will create EC2 instances in each VPC so that they can send data to each other later in the project, which gives us network activity to monitor.

Step 3 - Set up Logs

In this step, I will set up a way to track all inbound and outbound network traffic and set up a space that stores all the records using VPC Flow Logs.

Step 4 - Set IAM permissions for Logs

In this step, I will give VPC Flow Logs the permission to write logs and send them to CloudWatch and finish setting up your subnet's flow log because VPC Flow Logs doesn't have the permission to write logs and send them to CloudWatch.



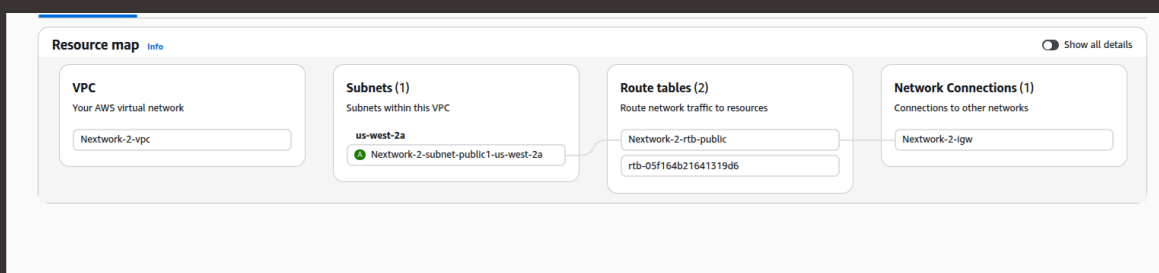
Multi-VPC Architecture

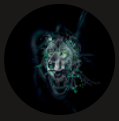
I started my project by launching two VPCs each with one public subnet and customised the CIDR blocks for both VPCs to ensure they are not the same.

The CIDR blocks for VPCs 1 and 2 are different (10.1.0.0/16 and 10.2.0.0/16 respectively). They have to be unique to prevent the IP addresses from overlapping which can create routing conflicts and connectivity issues.

I also launched EC2 instances in each subnet

My EC2 instances' security groups allow all ICMP traffic from all IP addresses. This is because we are going to ping the instance which requires the security group to allow ICMP traffic from all IP addresses.

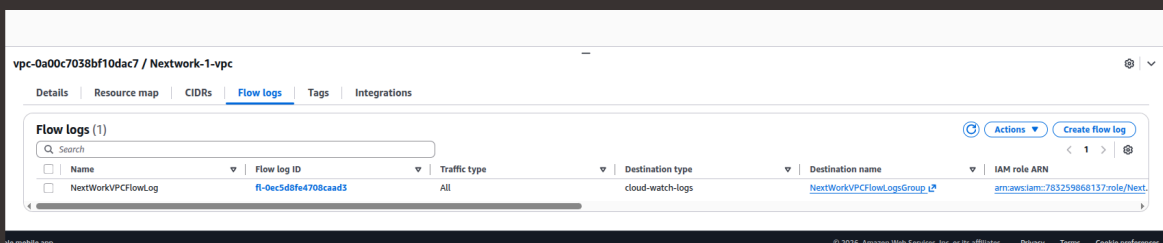


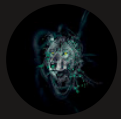


Logs

Logs are like a diary for your computer systems. They record everything that happens, from users logging in to errors popping up. It's the go-to place to understand what's going on with your systems, troubleshoot problems, and keep an eye on who's doing what.

Log groups are a big folder in AWS where you keep related logs together. Usually, logs from the same source or application will go into the same log group, BUT logs are also region-specific. This means log data gets created and saved in the region it was created, although you can use CloudWatch dashboards to bring together logs from different regions.



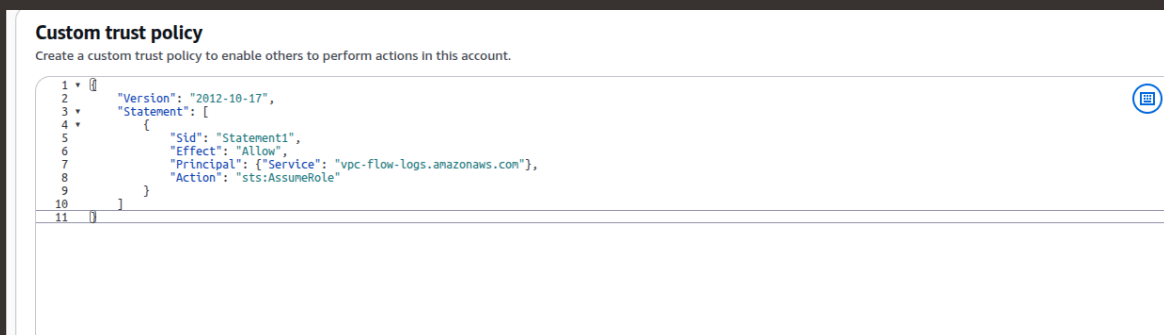


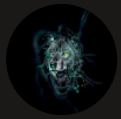
IAM Policy and Roles

I created an IAM policy because VPC Flow Logs by default don't have the permission to record logs and store them in the CloudWatch log group. Creating the policy makes sure that the VPC can now send log data to your log group

I also created an IAM role in order to assign the IAM policy that I created to the new role. The role is then assigned to VPC Flow Logs

A custom trust policy is used to very narrowly define who can use a role.





In the second part of my project...

Step 5 - Ping testing and troubleshooting

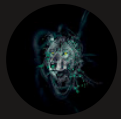
In this step, I will set up VPC peering and generate network traffic by trying to get instance in VPC 1 to send a message to the instance in VPC 2

Step 6 - Set up a peering connection

In this step, I will set up a connection link between the two VPCs which will allow the two VPCs to communicate via their private IP addresses.

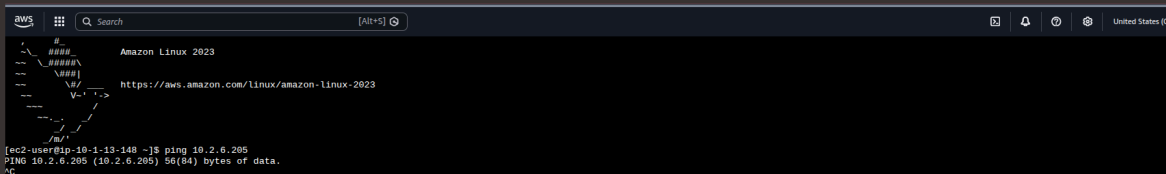
Step 7 - Analyze flow logs

In this step, I will review the flow logs recorded about VPC 1's public subnet and analyse the flow logs to get some insights



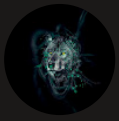
Connectivity troubleshooting

My first ping test between my EC2 instances had no replies, which means there is no VPC peering connection that directly connects VPCs 1 and 2.



```
aws
[ec2-user@ip-10-1-13-149 ~]$ ping 10.2.6.205
PING 10.2.6.205 (10.2.6.205) 56(84) bytes of data:
^C
```

I could receive ping replies if I ran the ping test using the other instance's public IP address, which means Instance 2 is correctly configured to respond to ping requests, and Instance 1 can actually communicate with Instance 2 if it traffic goes across the public internet!



Connectivity troubleshooting

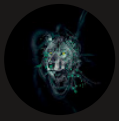
Looking at VPC 1's route table, I identified that the ping test with Instance 2's private address failed because there is no VPC peering which allows instances to communicate with their private IP addresses, the reason why we cannot ping to the private IP address.

To solve this, I set up a peering connection between my VPCs

I also updated both VPCs' route tables so that they allow VPC peering which allows communication of the two VPCs via their private IP addresses

The screenshot shows the AWS Route Table console for the route table 'rtb-0c303d6cfc27d59b3 / Nextwork-1-rtb-public'. It displays a list of routes with columns for Destination, Target, Status, Propagated, and Route Origin. There are three routes listed, all with a status of 'Active'.

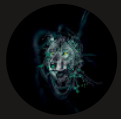
Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	lgw-00870c11924edc944	Active	No	Create Route
10.1.0.0/16	local	Active	No	Create Route Table
10.2.0.0/16	pcx-070524430b6e272ee	Active	No	Create Route



Connectivity troubleshooting

I received ping replies from Instance 2's private IP address! This means that instance 1 can communicate with instance 2 via the private IP address

```
lec2-user@ip-10-1-13-148 ~$ ping 10.2.6.205
PING 10.2.6.205 (10.2.6.205) 56(84) bytes of data:
64 bytes from 10.2.6.205: icmp_seq=1 ttl=127 time=1.39 ms
64 bytes from 10.2.6.205: icmp_seq=2 ttl=127 time=0.739 ms
64 bytes from 10.2.6.205: icmp_seq=3 ttl=127 time=0.799 ms
64 bytes from 10.2.6.205: icmp_seq=4 ttl=127 time=0.435 ms
64 bytes from 10.2.6.205: icmp_seq=5 ttl=127 time=0.408 ms
64 bytes from 10.2.6.205: icmp_seq=6 ttl=127 time=0.505 ms
64 bytes from 10.2.6.205: icmp_seq=7 ttl=127 time=0.500 ms
64 bytes from 10.2.6.205: icmp_seq=8 ttl=127 time=0.792 ms
64 bytes from 10.2.6.205: icmp_seq=9 ttl=127 time=0.488 ms
64 bytes from 10.2.6.205: icmp_seq=10 ttl=127 time=0.717 ms
64 bytes from 10.2.6.205: icmp_seq=11 ttl=127 time=0.459 ms
64 bytes from 10.2.6.205: icmp_seq=12 ttl=127 time=0.875 ms
64 bytes from 10.2.6.205: icmp_seq=13 ttl=127 time=0.588 ms
64 bytes from 10.2.6.205: icmp_seq=14 ttl=127 time=0.726 ms
64 bytes from 10.2.6.205: icmp_seq=15 ttl=127 time=0.492 ms
64 bytes from 10.2.6.205: icmp_seq=16 ttl=127 time=0.482 ms
64 bytes from 10.2.6.205: icmp_seq=17 ttl=127 time=0.463 ms
64 bytes from 10.2.6.205: icmp_seq=18 ttl=127 time=0.472 ms
64 bytes from 10.2.6.205: icmp_seq=19 ttl=127 time=0.848 ms
64 bytes from 10.2.6.205: icmp_seq=20 ttl=127 time=0.429 ms
```



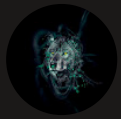
Analyzing flow logs

Flow logs tell us about the bytes that were sent successfully from a given IP address and to a destination address using a TCP protocol of a given port and whether the packets were accepted or denied

For example, the flow log I've captured tells us 344 bytes of data were sent successfully from the IP address 147.185.132.233 to 10.1.13.148 using TCP protocol on port 22, with 44 packets transferred and the traffic was allowed ("ACCEPT").

There are older events to load. [Load more.](#)

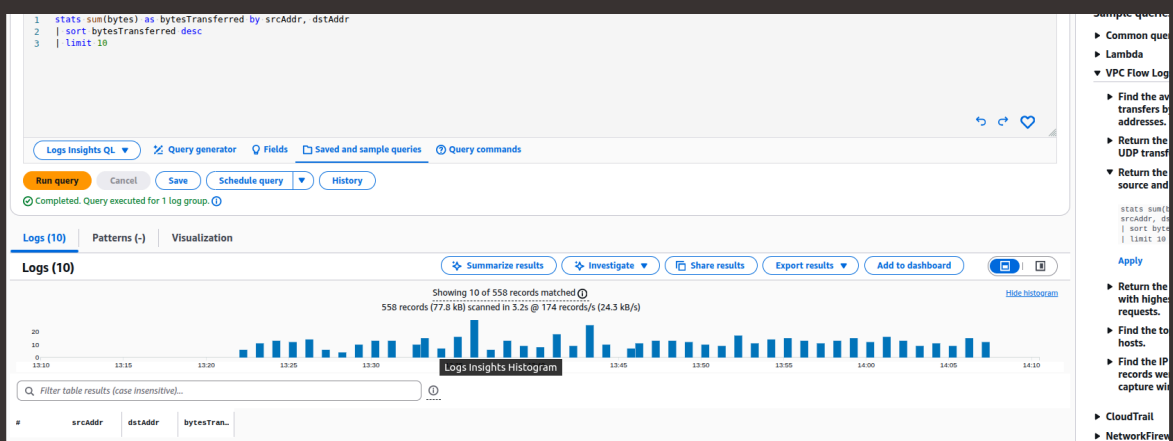
▼	2020-05-09T13:50:21.000Z	2	783259868137	en1-0a2091d4c6e376e05	147.185.132.233	10.1.13.148	40191	34435	0	1	44	1778334021	1778334089	REJECT	OK
		2	783259868137	en1-0a2091d4c6e376e05	147.185.132.233	10.1.13.148	40191	34435	0	1	44	1778334021	1778334089	REJECT	OK



Logs Insights

Logs Insights is CloudWatch feature that analyzes your logs. In Log Insights, you use queries to filter, process and combine data to help you troubleshoot problems or better understand your network traffic!

I ran the query Top 10 byte transfers by source and destination IP addresses . This query analyzes the top 10 biggest data transfers between IP addresses in the network! It helps find out which resources are moving the most data, which uncovers the busiest traffic routes in the network.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

